INSTITUTO FEDERAL DO RIO GRANDE DO NORTE · Campus Pau dos Ferros
Tecnologia em Análise e Desenvolvimento de Sistemas · Desenvolvimento de Sistemas Corporativos · 2026.2
Prof. Jeferson Queiroga

Projeto API REST — Sistema de Chamados do IFRN

Projeto individual

---

1. Apresentação

Você vai desenvolver, **individualmente**, uma API REST com Spring Boot para um sistema de chamados de suporte do IFRN. O tema é simples de propósito: o foco está em aplicar bem a arquitetura em camadas, a persistência com JPA e a entrega com Docker vistas na disciplina.

Ao final, qualquer pessoa deve conseguir clonar seu repositório, rodar `docker compose up` e usar a API sem configurar nada na máquina além do Docker.

2. Cenário

Hoje, quando um projetor para de funcionar ou o Wi-Fi do laboratório cai, o servidor ou aluno avisa o setor responsável por WhatsApp, e-mail ou pessoalmente. Não há registro, prioridade nem histórico do que foi feito.

A API vai centralizar esses pedidos. Um **solicitante** abre um chamado informando o problema, o local e a categoria. O chamado vai para o **setor** responsável (por exemplo, COTIC para TI ou Manutenção para infraestrutura), onde um **técnico** o assume, registra o andamento e o encerra. O solicitante acompanha tudo pelo histórico do chamado.

3. Escopo

O sistema tem cinco entidades. Os atributos abaixo são o mínimo; você pode acrescentar outros se justificar.

Entidade	Atributos mínimos	Relacionamentos
Setor	id, nome, sigla (única)	tem várias categorias e vários técnicos
Categoria	id, nome, descrição	pertence a um setor
Usuario	id, nome, email (único), matrícula, tipo (SOLICITANTE ou TECNICO)	técnico pertence a um setor
Chamado	id, título, descrição, local, prioridade, status, dataAbertura, dataFechamento, solução	tem um solicitante, uma categoria e, opcionalmente, um técnico
Interacao	id, texto, dataHora	pertence a um chamado e tem um autor (usuário)

Prioridade assume BAIXA, MEDIA ou ALTA. Status assume ABERTO, EM_ATENDIMENTO, RESOLVIDO ou CANCELADO.

Ciclo de vida do chamado

`ABERTO` → `EM_ATENDIMENTO` → `RESOLVIDO`, e `ABERTO` → `CANCELADO`. Nenhuma outra transição é permitida.

Regras de negócio

Cada regra deve ficar na camada de **Service**, nunca no Controller.

1. Todo chamado nasce com status ABERTO, `dataAbertura` preenchida pelo sistema e prioridade MEDIA quando não informada.
2. Apenas usuários do tipo SOLICITANTE podem abrir chamados.
3. Um técnico só pode assumir um chamado ABERTO cuja categoria pertença ao seu setor. Ao assumir, o status muda para EM_ATENDIMENTO.
4. Somente o técnico atribuído pode resolver o chamado, e a `solução` é obrigatória. Ao resolver, `dataFechamento` é preenchida.
5. Somente o próprio solicitante pode cancelar, e apenas enquanto o chamado estiver ABERTO.
6. Chamados RESOLVIDO ou CANCELADO não aceitam alterações nem novas interações.
7. Apenas o solicitante e o técnico atribuído podem registrar interações no chamado.
8. Não é permitido excluir setor ou categoria que tenha chamados vinculados.

Como o projeto não usa autenticação, o usuário que executa a ação é informado no corpo da requisição (por exemplo, `solicitanteId` ou `tecnicoId`).

4. Requisitos funcionais

A API deve expor, no mínimo, os endpoints abaixo. Use os códigos HTTP indicados.

Cadastros básicos

Recurso	Endpoints	Observação
Setores	`POST`, `GET` (lista e por id), `PUT`, `DELETE` em `/setores`	regra 8 no DELETE
Categorias	`POST`, `GET` (lista e por id), `PUT`, `DELETE` em `/categorias`	`GET /setores/{id}/categorias` lista as de um setor
Usuários	`POST`, `GET` (lista e por id), `PUT` em `/usuarios`	email duplicado retorna 409

Chamados

Método e rota	O que faz	Sucesso
`POST /chamados`	abre um chamado	201
`GET /chamados`	lista paginada com filtros (ver Paginação e filtros)	200
`GET /chamados/{id}`	detalha o chamado com suas interações	200
`GET /usuarios/{id}/chamados`	chamados abertos por um solicitante ou atribuídos a um técnico	200
`PATCH /chamados/{id}/assumir`	técnico assume o chamado	200
`PATCH /chamados/{id}/resolver`	técnico resolve informando a solução	200
`PATCH /chamados/{id}/cancelar`	solicitante cancela	200
`POST /chamados/{id}/interacoes`	registra uma interação	201

Paginação e filtros

Toda rota de listagem (`GET` que retorna coleção) deve ser paginada e aceitar filtros opcionais por query string.

- Paginação com `Pageable` do Spring Data: parâmetros `page`, `size` e `sort` (exemplo: `?page=0&size=10&sort=dataAbertura,desc`). Tamanho padrão de 10 itens e máximo de 50.
- A resposta traz os itens e os metadados da página: número da página, tamanho, total de elementos e total de páginas.
- Filtros combináveis entre si; filtro não informado é ignorado.

Listagem	Filtros mínimos
`GET /setores`	`nome` (contém, sem diferenciar maiúsculas)
`GET /categorias`	`nome`, `setorId`
`GET /usuarios`	`nome`, `tipo`, `setorId`
`GET /chamados`	`status`, `prioridade`, `setorId`, `categoriaId`, `solicitanteId`, `tecnicoId`, período de abertura (`dataInicio` e `dataFim`)
`GET /usuarios/{id}/chamados`	`status`

Exemplo:

```
GET /chamados?status=ABERTO&setorId=1&prioridade=ALTA&page=0&size=10&sort=dataAbertura,desc
```

Respostas de erro

- 400 para dados inválidos (campo obrigatório vazio, email mal formatado).
- 404 para recurso inexistente.
- 409 para violação de regra de negócio (transição de status inválida, técnico de outro setor).
- Todo erro devolve um JSON padronizado com `timestamp`, `status`, `mensagem` e `caminho`, gerado por um `@RestControllerAdvice`.

5. Requisitos técnicos obrigatórios

Stack

- Java 21 e Spring Boot 4, com Maven.
- Spring Web, Spring Data JPA e PostgreSQL.
- Lombok é permitido.

Arquitetura em camadas

- Pacotes separados para `controller`, `service`, `repository` e `model` (entidades).
- Controller só recebe a requisição, chama o Service e devolve a resposta. Nenhuma regra de negócio nele.
- Service concentra as regras da seção 3 e lança exceções próprias (por exemplo, `RegraNegocioException`).
- Injeção de dependência por construtor, sem `@Autowired` em atributo.
- Repository estende `JpaRepository`; filtros com métodos derivados ou `@Query`.

Persistência

- Relacionamentos mapeados com `@ManyToOne` e `@OneToMany`, enums com `@Enumerated(EnumType.STRING)`.

- Configuração do banco por variáveis de ambiente, nunca com senha fixa no `application.properties`.
- Esquema do banco versionado com **migrations Flyway**, em `src/main/resources/db/migration`. O Hibernate não cria tabelas: use `spring.jpa.hibernate.ddl-auto=validate`.
- Uma migration por mudança, nomeada em sequência (`V1__criar_tabelas.sql`, `V2__inserir_dados_iniciais.sql`, ...). Migration já aplicada nunca é editada: correções entram em uma nova versão.
- Dados iniciais também em migration: ao menos 2 setores, 4 categorias e 4 usuários.

Para estudar: migrations não foram aprofundadas em aula. Pesquise como o Flyway registra as versões na tabela `flyway_schema_history`, o que acontece quando o checksum de uma migration aplicada muda e por que `ddl-auto=update` não é usado em produção. Explique isso em uma seção curta do `README.md`.

Docker

- `Dockerfile` multi-stage (build com Maven, execução com JRE).
- `docker-compose.yml` subindo a API e o PostgreSQL, com volume para os dados.
- `docker compose up` deve deixar a API respondendo em `http://localhost:8080` sem nenhum passo manual.

Repositório

- Público no GitHub, com histórico de commits ao longo do desenvolvimento (não um único commit no final).
- `README.md` com descrição do projeto, como executar e exemplos de requisição.
- Uma coleção do Postman ou Insomnia (ou arquivo `.http`) com todas as requisições da seção 4.

6. Entrega

- **Como entregar:** link do repositório público no Google Sala de Aula. Será avaliado o último commit na `main`.